I am rethinking my project. How can I make stuff much more aqurate, can I somehow learn about NLP and apply that to have NL2SQL? Can I make use of existing MCP Servers? What approaches have other people tooken? What are the best open source LLM's for eahc of my agents?

Based on my research, here's a comprehensive analysis of how to dramatically improve your NL2SQL project:[1] [2] [3]

State-of-the-Art Approaches (2025-2026)

Multi-Agent Frameworks (What Top Systems Use)

The **MAC-SQL framework** is currently state-of-the-art, achieving **59.59% execution accuracy** on the BIRD benchmark. It uses three specialized agents:[4] [1]

1. **Selector Agent** - Schema simplification (similar to your Agent #2)
2. **Decomposer Agent** - Query decomposition with Chain-of-Thought (similar to your Agent #1)
3. **Refiner Agent** - SQL validation and correction (what you need for Agent #4)

Your architecture is already aligned with SOTA approaches![1] [4]

Best Open-Source LLMs for Each Agent (2026)

Agent #1: Question Decomposer

Best Choice: DeepSeek-V3[5]

- Excels at reasoning and algorithm thinking
- Strong at breaking down complex problems
- 671B parameters (37B active via MoE)
- **Free and open-source**

Alternative: Qwen3-Coder[6] [5]

- 480B parameters (35B active)
- 256K context window

- Better for clean, direct solutions

Agent #2: Schema Scout

Best Choice: GLM-4.6[7]

- 200K token context window (can handle massive schemas)
- Strong agentic reasoning
- Optimized for tool use and structured tasks

Agent #3: SQL Architect

Best Choice: Qwen3-Coder[5] [6]

- State-of-the-art code generation
- Production-ready SQL output
- Strong at structured output tasks
- Clean, modular code generation

Alternative: DeepSeek-V3[5]

- Deeper algorithmic reasoning
- Better debugging capabilities
- More explanatory (good for learning)

Accuracy Improvements: What Works

1. Schema Linking with Sample Data[8]

TCSR-SQL achieved +13.7% accuracy by including table content (sample rows) in prompts - **you already do this!** ✅

2. Chain-of-Thought Reasoning[9]

STaR-SQL uses step-by-step reasoning for SQL generation:

```
1. Identify required tables
2. Determine JOIN conditions
3. Build WHERE clause
4. Add aggregations
5. Construct final SQL
```

Your Agent #3 already does this! ✅

3. Self-Correction with Execution Feedback[10]

Reliable Text-to-SQL (RTS) framework:

- Execute generated SQL
- If error, feed error message back to LLM
- Let it self-correct
- **This should be your Agent #4!** ⚠

4. Abstention (Knowing When to Give Up)[10]

State-of-the-art systems now include **confidence thresholds**:

- If confidence < 0.7 → ask user for clarification
- Better to admit uncertainty than hallucinate
- **You already track confidence!** ✅

Leverage Existing MCP Servers

Database MCP Servers You Can Use[11] [12]

MCP Server	Best For	Why Use It
Supabase MCP [11]	PostgreSQL with auth	Production-ready, handles RLS policies
SQLite MCP [11] [12]	Local dev	Safe experimentation, no prod risk
MindsDB MCP [11]	Multi-DB queries	Federated queries across databases
Prisma MCP [11]	Schema + data + migrations	Schema-aware queries
PostgreSQL MCP [12]	Direct access	Fast, reliable

How to integrate:

```
# Instead of building your own MCP server, USE existing ones
from mcp import Client

client = Client("sqlite")  # or "supabase", "postgresql"
result = client.query("SELECT * FROM products")
```

Benefits:

- ✅ Battle-tested by thousands of developers
- ✅ Automatic schema introspection
- ✅ Security built-in
- ✅ Active maintenance

🧠 NLP Techniques to Apply

1. Named Entity Recognition (NER)

Identify entities in questions:

```
from transformers import pipeline

ner = pipeline("ner", model="dbmdz/bert-large-cased-finetuned-conll03-english")
entities = ner("Show me products from Trek brand")
# → [{"word": "Trek", "entity": "ORG"}]
```

2. Semantic Similarity for Column Matching

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer('all-MiniLM-L6-v2')

question_embedding = model.encode("customer names")
column_embeddings = model.encode(["first_name", "last_name", "email"])

# Find most similar columns
```

3. Query Classification

Train a classifier for query types:

- SELECT vs COUNT vs AGGREGATION vs JOIN
- **You already do this with LLM!** ✅

🚀 Practical Improvements for Your Project

Immediate Wins (This Week)

1. **Switch to DeepSeek-V3 or Qwen3-Coder** for all agents
   - Both are free, open-source, and SOTA for code[6] [5]
2. **Add Self-Correction Loop** (Agent #4)

   ```
   def execute_and_refine(sql, max_retries=3):
       for i in range(max_retries):
           result = execute(sql)
           if result.success:
               return result
           # Feed error back to LLM
           sql = llm.refine(sql, error=result.error)
       return None
   ```

3. **Use Existing PostgreSQL MCP Server**[12]
    - Replace your custom DatabaseMCPServer
    - Get schema introspection for free

Medium-Term (Next 2 Weeks)

4. **Add Embedding-Based Column Matching**
    - Use sentence-transformers for semantic search
    - Match "customer names" → ["first_name", "last_name"]
5. **Implement Confidence Thresholds**

```
if confidence < 0.7:
    return "I'm not confident. Did you mean...?"
```

6. **Create a Golden Dataset**
    - 50 question-SQL pairs for your bike store DB
    - Use for testing and fine-tuning

Advanced (Month 2+)

7. **Fine-tune on Your Domain**
    - Use MAC-SQL approach[4] [1]
    - Fine-tune DeepSeek-V3 or Qwen3 on your bike store queries
8. **Add Query Decomposition**
    - For complex queries: "Which brand has most expensive bikes in California stores?"
    - Break into sub-queries, solve sequentially

Benchmark Your System

State-of-the-art systems report:

- **80-90% execution accuracy** on Spider benchmark[13]
- **60% on BIRD benchmark** (harder, real-world DBs)[1]

Track these metrics:

- Execution accuracy (does SQL run?)
- Exact match accuracy (perfect SQL?)
- Valid SQL ratio (syntax correct?)

✅ Your Current Strengths

You're already doing many SOTA practices:

- ✅ Multi-agent architecture (like MAC-SQL)[4] [1]
- ✅ Sample data for grounding[8]
- ✅ Chain-of-Thought reasoning[9]
- ✅ Confidence tracking[10]
- ✅ Schema simplification

You're closer to state-of-the-art than you think!

Recommended Next Steps

1. **Today:** Switch to **DeepSeek-V3** (free, 671B params, SOTA reasoning)[5]
2. **This week:** Add self-correction loop (Agent #4)
3. **Next week:** Replace custom MCP with existing PostgreSQL/SQLite MCP[12]
4. **Month 1:** Add semantic column matching with embeddings
5. **Month 2:** Fine-tune on 50+ bike store query examples

Focus on execution accuracy, not perfection. Even 90% accuracy requires human review - build for that workflow.[14]

⁂

1. http://arxiv.org/pdf/2312.11242.pdf
2. https://arxiv.org/pdf/2412.05208.pdf
3. http://arxiv.org/pdf/2406.08426v3.pdf
4. https://arxiv.org/html/2312.11242v2
5. https://www.secondtalent.com/resources/qwen-3-vs-deepseek-v3-best-chinese-llm-for-code-generation/
6. https://sourceforge.net/software/compare/DeepSeek-V3-vs-Qwen3-Coder/
7. https://www.datacamp.com/blog/top-open-source-llms
8. http://arxiv.org/pdf/2407.01183.pdf
9. https://arxiv.org/pdf/2502.13550.pdf
10. https://arxiv.org/html/2501.10858v1
11. https://www.builder.io/blog/best-mcp-servers-2026
12. https://mcpplaygroundonline.com/blog/awesome-mcp-servers
13. https://promethium.ai/guides/text-to-sql-basics-benefits/
14. https://towardsdatascience.com/why-90-accuracy-in-text-to-sql-is-100-useless/
15. http://arxiv.org/pdf/2412.10138.pdf
16. https://arxiv.org/pdf/2402.16347.pdf

17. https://textql.com/blog/ana-small/
18. https://blogs.oracle.com/cloud-infrastructure/oracle-wins-archer-nl2sql-challenge
19. https://www.bytebase.com/blog/top-text-to-sql-query-tools/
20. https://arxiv.org/html/2410.22925v1
21. https://ai.meta.com/research/publications/cwm-an-open-weights-llm-for-research-on-code-generation-with-world-models/
22. https://contextual.ai/blog/open-sourcing-the-best-local-text-to-sql-system/
23. https://aclanthology.org/2025.findings-emnlp.1031.pdf
24. http://biorxiv.org/lookup/doi/10.64898/2026.01.18.700012
25. https://www.mdpi.com/2075-5309/16/3/485
26. https://academic.oup.com/database/article/doi/10.1093/database/baaf088/8426097
27. http://biorxiv.org/lookup/doi/10.1101/2025.09.10.675105
28. https://www.semanticscholar.org/paper/50953759497b3889ef0def05ef0be8c5615e0de7
29. https://arxiv.org/abs/2510.25726
30. https://www.tib-op.org/ojs/index.php/ocp/article/view/3161
31. https://www.spiedigitallibrary.org/conference-proceedings-of-spie/13955/3092490/Design-of-UAV-flight-trajectory-monitoring-system-based-on-low/10.1117/12.3092490.full
32. http://biorxiv.org/lookup/doi/10.1101/2025.09.02.673797
33. https://www.jisem-journal.com/index.php/journal/article/view/13643
34. http://arxiv.org/pdf/1902.06288.pdf
35. https://arxiv.org/pdf/2503.23278.pdf
36. https://arxiv.org/pdf/2501.14387.pdf
37. http://arxiv.org/pdf/1705.05343.pdf
38. http://arxiv.org/pdf/2412.02792.pdf
39. http://arxiv.org/pdf/2209.13643.pdf
40. https://arxiv.org/pdf/2305.08468.pdf
41. https://arxiv.org/pdf/2106.07351.pdf
42. https://mcpmarket.com/daily/top-mcp-server-list-january-31-2026
43. https://www.k2view.com/blog/awesome-mcp-servers
44. https://www.datacamp.com/blog/top-remote-mcp-servers
45. https://github.com/wong2/awesome-mcp-servers
46. https://arxiv.org/abs/2312.11242